

Algoritmi e Strutture Dati

Insiemi e dizionari Riassunto finale

Alberto Montresor

Università di Trento

2026/08/15

This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License.



Insiemi e dizionari

Insiemi

- Collezione di oggetti

Dizionari

- Associazioni chiave-valore

Implementazione

- Molte delle strutture dati viste finora
- Vantaggi e svantaggi

Insiemi realizzati con vettori booleani

Insieme

- Interi $0 \dots m - 1$
- Collezione di m oggetti memorizzati in un vettore

Rappresentazione

- Vettore booleano di m elementi

Vantaggi

- Notevolmente semplice
- Efficiente verificare se un elemento appartiene all'insieme

Svantaggi

- Memoria occupata $\mathcal{O}(m)$, indipendente dalle dimensioni effettive
- Alcune operazioni inefficienti – $\mathcal{O}(m)$

Insiemi realizzati con vettori booleani

SET (vettore booleano)

boolean[] *V*

int *size*

int *dim*

SET **Set**(**int** *m*)

SET *t* = **new** SET

t.size = 0

t.dim = *m*

t.V = **new boolean**[0...*m* - 1] =
 {false}

return *t*

boolean **contains**(**int** *x*)

if $0 \leq x \leq \text{dim} - 1$ **then**

return *V*[*x*]

else

return false

int **size**()

return *size*

insert(**int** *x*)

if $0 \leq x \leq \text{dim} - 1$ **then**

if not *V*[*x*] **then**

size = *size* + 1

V[*x*] = **true**

remove(**int** *x*)

if $0 \leq x \leq \text{dim} - 1$ **then**

if *V*[*x*] **then**

size = *size* - 1

V[*x*] = **false**

Insiemi realizzati con vettori booleani

SET (vettore booleano)

SET union(SET *A*, SET *B*)

```
int newsize = max(A.dim, B.dim)
```

```
SET C = Set(newsize)
```

```
for i = 0 to A.dim - 1 do
```

```
    if A.contains(i) then
```

```
        C.insert(i)
```

```
for i = 0 to B.dim - 1 do
```

```
    if B.contains(i) then
```

```
        C.insert(i)
```

```
return C
```

Insiemi realizzati con vettori booleani

SET (vettore booleano)

SET difference(SET *A*, SET *B*)

```
SET C = Set(A.dim)
for i = 0 to A.dim - 1 do
    if A.contains(i) and not B.contains(i) then
        C.insert(i)
return C
```

SET intersection(SET *A*, SET *B*)

```
int newsize = min(A.dim, B.dim)
SET C = Set(newsize)
for i = 0 to min(A.dim, B.dim) - 1 do
    if A.contains(i) and B.contains(i) then
        C.insert(i)
return C
```

BitSet

Java - class `java.util.BitSet`

Metodo	Operaz.	Metodo	Operaz.
<code>void and(BitSet set)</code>	Intersection	<code>void clear(int i)</code>	Remove
<code>void or(BitSet set)</code>	Union	<code>void set(int i)</code>	Insert
<code>int cardinality()</code>	Set size	<code>boolean get(int i)</code>	Contains

C++ STL

- `std::bitset` – Struttura dati bitset con dimensione fissata nel template al momento della compilazione.
- `std::vector<bool>` – Specializzazione di `std::vector` per ottimizzare la memorizzazione, dimensione dinamica.

Insiemi realizzati con liste / vettori non ordinati

Costo operazioni

- Operazioni di ricerca, inserimento e cancellazione: $\mathcal{O}(n)$
- Operazioni di inserimento (assumendo assenza): $\mathcal{O}(1)$
- Operazioni di unione, intersezione e differenza: $\mathcal{O}(nm)$

SET **difference**(**SET** A , **SET** B)

SET C = **Set**()

foreach $s \in A$ **do**

```
└   if not  $B$ .contains( $s$ ) then
    └    $C$ .insert( $s$ )
```

return C

Insiemi realizzati con liste / vettori ordinati

LIST intersection(**LIST** *A*, **LIST** *B*)

LIST *C* = **Set**()

Pos *pos_a* = *A*.head()

Pos *pos_b* = *B*.head()

while

not *A*.finished(*pos_a*) **and** **not** *B*.finished(*pos_b*)

do

if *A*.read(*pos_a*) == *B*.read(*pos_b*) **then**

C.insert(*C*.tail(), *A*.read(*pos_a*))

pos_a = *A*.next(*pos_a*)

pos_b = *B*.next(*pos_b*)

else if *A*.read(*pos_a*) < *B*.read(*pos_b*) **then**

pos_a = *A*.next(*pos_a*)

else

pos_b = *B*.next(*pos_b*)

return *C*

Costo operazioni

- Ricerca:
 - $\mathcal{O}(n)$ (liste)
 - $\mathcal{O}(\log n)$ (vettori)
- Inserimento e cancellazione
 - $\mathcal{O}(n)$
- Unione, intersezione e differenza:
 - $\mathcal{O}(n)$

Insiemi – Strutture dati complesse

Alberi bilanciati

- Ricerca, inserimento, cancellazione: $\mathcal{O}(\log n)$
- Iterazione: $\mathcal{O}(n)$
- Con ordinamento
- Implementazioni:
 - Java TreeSet
 - Python OrderedSet
 - C++ STL set

Hash table

- Ricerca, inserimento, cancellazione: $\mathcal{O}(1)$
- Iterazione: $\mathcal{O}(m)$
- Senza ordinamento
- Implementazioni:
 - Java HashSet
 - Python set
 - C++ STL unordered_set

Insiemi – Riassunto

	contains lookup	insert	remove	min	foreach	Ordine
Vettore booleano	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(m)$	$\mathcal{O}(m)$	Sì
Lista non ordinata	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	No
Lista ordinata	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	Sì
Vettore ordinato	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	Sì
Alberi bilanciati	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	Sì
Hash (Mem. interna)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(m)$	$\mathcal{O}(m)$	No
Hash (Mem. esterna)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(m+n)$	$\mathcal{O}(m+n)$	No

$m \equiv$ dimensione del vettore o della tabella hash

Python – List

Operazione	Caso medio	Caso pessimo ammortizzato
<code>L.copy()</code> Copy	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>L.append(x)</code> Append	$\mathcal{O}(1)$	$\mathcal{O}(1)$
<code>L.insert(i, x)</code> Insert	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>L.remove(x)</code> Remove	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>L[i]</code> Index	$\mathcal{O}(1)$	$\mathcal{O}(1)$
<code>for x in L</code> Iterator	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>L[i:i+k]</code> Slicing	$\mathcal{O}(k)$	$\mathcal{O}(k)$
<code>L.extend(s)</code> Extend	$\mathcal{O}(k)$	$\mathcal{O}(k)$
<code>x in L</code> Contains	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>min(L),</code> <code>max(L)</code> Min, Max	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>len(L)</code> Get length	$\mathcal{O}(1)$	$\mathcal{O}(1)$

$$n = \text{len}(L)$$

Python – Set

Operazione		Caso medio	Caso pessimo
<code>x in S</code>	Contains	$\mathcal{O}(1)$	$\mathcal{O}(n)$
<code>for x in S</code>	Iterator	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>S.add(x)</code>	Insert	$\mathcal{O}(1)$	$\mathcal{O}(n)$
<code>S.remove(x)</code>	Remove	$\mathcal{O}(1)$	$\mathcal{O}(n)$
<code>S T</code>	Union	$\mathcal{O}(n + m)$	$\mathcal{O}(n \cdot m)$
<code>S&T</code>	Intersection	$\mathcal{O}(\min(n, m))$	$\mathcal{O}(n \cdot m)$
<code>S-T</code>	Difference	$\mathcal{O}(n)$	$\mathcal{O}(n \cdot m)$

$$n = \text{len}(S), m = \text{len}(T)$$